PROGRESS UPDATE · JUNE 2026

Test Fixture GUI

Bench control software for the sensor-team test fixture

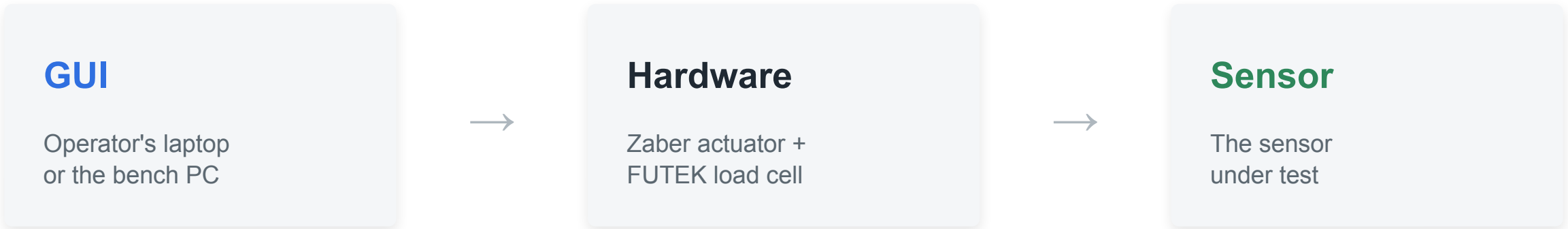
EM · Calibration · Manual · Shear · Fatigue

Jacqueline Chuang

OVERVIEW

What it is

A single browser-based app an operator uses at the bench to run the sensor team's mechanical tests - driving a precision actuator into a sensor while reading a load cell, with every test, safety check, and result in one place.



Closed-loop control: the GUI moves the actuator based on the live load-cell reading.

STATUS

Where the project is

5

test workflows

EM, Calibration, Manual, Shear, and Fatigue

6

safety systems

Force, spike, travel, disconnect, snap-to, single-owner reader

100+

user stories

Across 11 features, with a formal test plan underway

- MVP built and running end to end - every test type completes.
- Packaged as a double-clickable desktop app on Mac and Windows.

PART 1

What I built

CAPABILITIES

Five test workflows

1. **EM** — Press the sensor and read its response against pressure, automated across repeated runs.
2. **Calibration / Fuji** — Fuji-film calibration with an operator-set extrusion distance and a fine manual jog.
3. **Manual** — Free actuator control by distance or force, with live force recording for exploratory tests.
4. **Shear** — Load-cell-only capture - the operator applies force by hand while the GUI records it.
5. **Fatigue** — Cyclical loading on a sine or square waveform for thousands of cycles, with a live preview.

DEMO

Fatigue testing window

- **Live waveform preview**

  Sine or square, redrawn as the bounds and frequency change.

- **Whole-number force bounds**

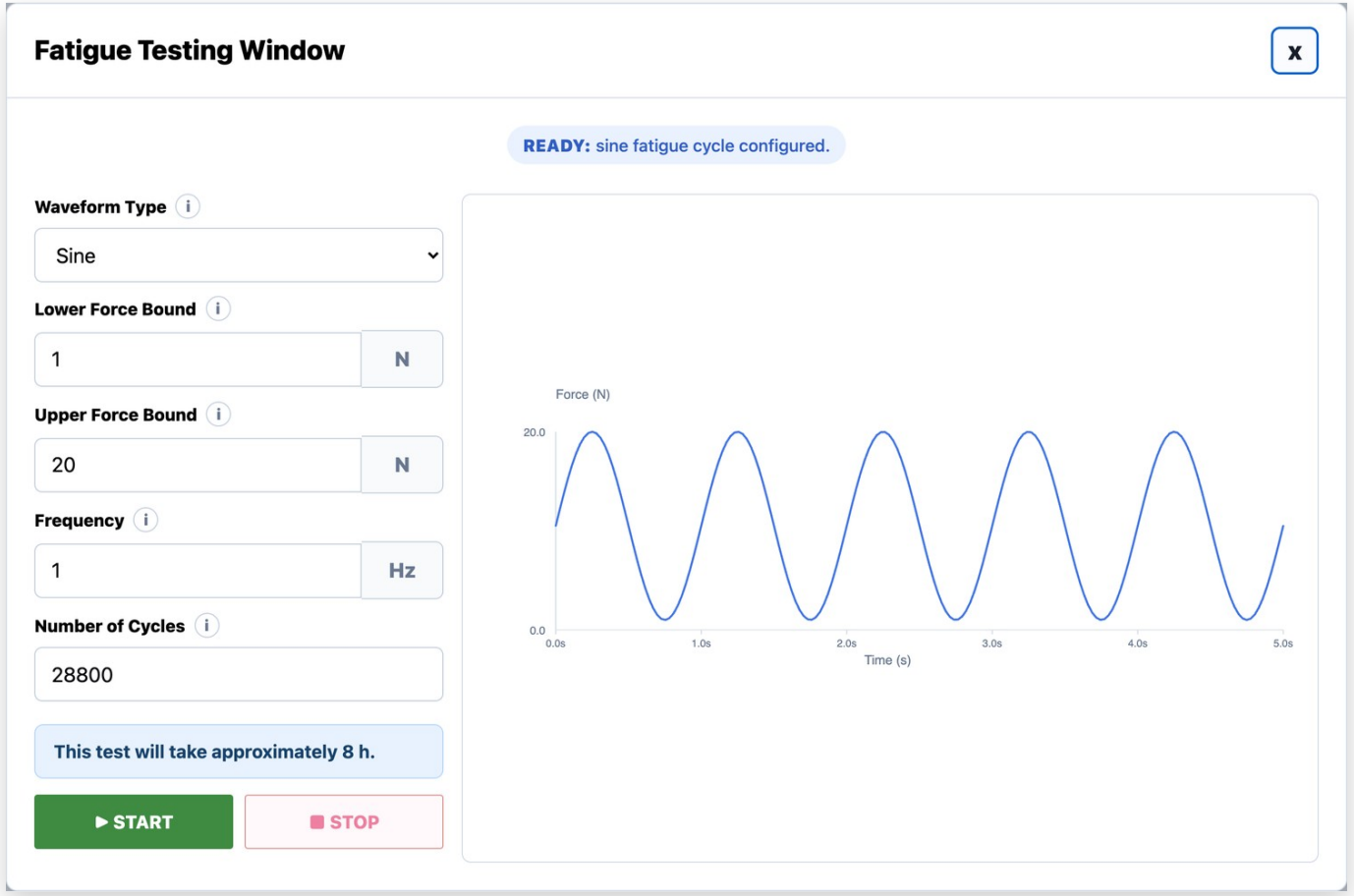
  Lower and upper force, snapped into the safe range on entry.

- **Cycle count + duration**

  Shows the estimated test length - here, ~8 hours for 28,800 cycles.

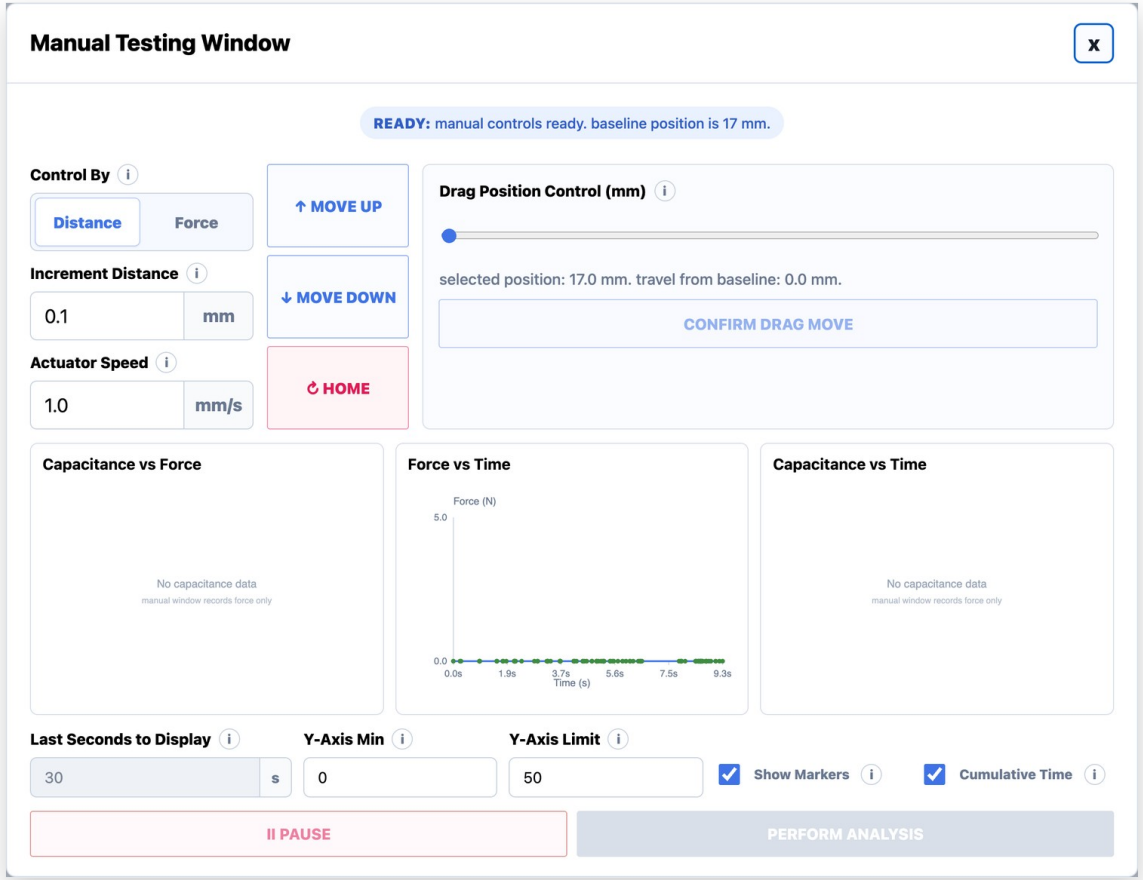
- **Force-spike protection**

  Stops safely if the load ever jumps beyond the configured swing.



DEMO

Manual testing window



- **Distance or force control**

  Jog by a set distance, or drive until the load cell reads a target force.

- **Compress and decompress**

  Closed-loop in both directions - press to a force, then release back to one.

- **Continuous force recording**

  Force vs time records from the moment the window opens.

- **Editable while moving**

  Graph controls stay live mid-move; motion controls lock for safety.

DEMO

EM test & run management

- **Live state machine**

  Every run moves through a strict set of states, each one timestamped in the status log.

- **Automated multi-run**

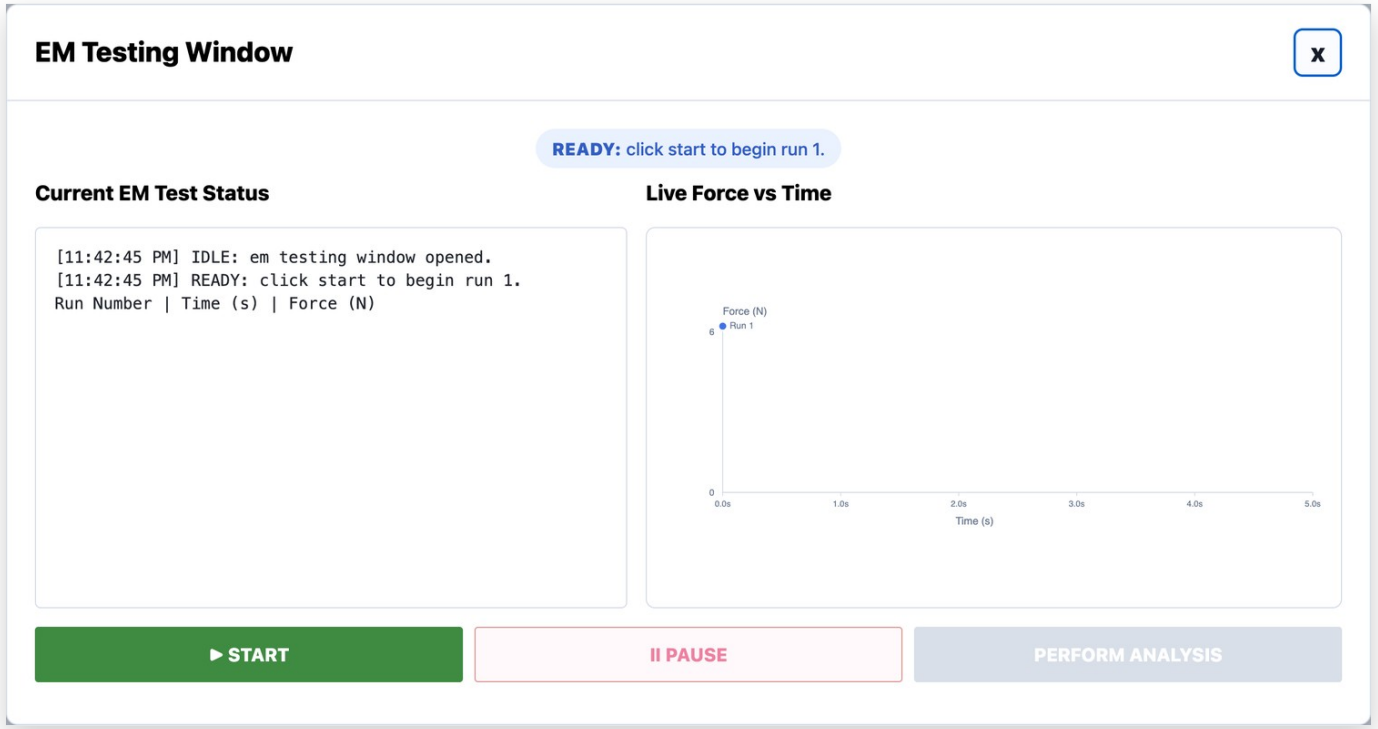
  Runs the configured presses back to back, auto-pausing so the sensor can be reset.

- **Redo without data loss**

  A redo creates a new superseding run - the original is never overwritten or deleted.

- **Homes first, every time**

  Drives to the true baseline before pressing, never starting from an unknown spot.



DETAIL

Calibration & Fuji film

1

Operator-set extrusion distance

A new field controls how far the actuator extrudes for the Fuji-film press, capped at 12 mm and remembering the last value used.

2

Fine manual jog

Step the actuator in small increments to fine-tune position before a calibration press.

3

Press to a calibrated target

The Fuji press drives down until the load cell reaches its target force, then stops.

4

Reliable jog state

Fixed an intermittent bug where the window thought the stage was still moving after a jog finished.

SAFETY

Built into every test

Force ceiling

A hard over-force limit stops the actuator before the load cell can be overloaded.

Spike detection

A sudden force jump - metal-on-metal contact - halts motion immediately.

Travel limits

The stage never drives into its mechanical end stop.

Disconnect handling

Losing the actuator or load cell mid-test stops safely and invalidates the run.

Snap-to input limits

Out-of-range entries snap to the nearest limit and say which limit they hit.

Single-owner load cell

Exactly one reader touches the device, so it can't be corrupted mid-session.

PART 2

Under the hood

ARCHITECTURE

How it's built

Browser GUI

Vanilla HTML / CSS / JS - one file per test tab (em.js, manual.js, fatigue.js, shear.js, calibration.js), plus shared.js and main.js

↓ *JSON over local HTTP · the UI polls for live status*

Local Python server + test engine

ThreadingHTTPServer (server.py) and run_engine.py - one background thread per test, polled for live status, a single-owner load-cell reader, and a simulation fallback

↓ *serial commands · .NET driver calls*

Hardware drivers

Zaber actuator over serial (zaber-motion) and the FUTEK load cell over .NET (pythonnet, Windows only)

Packaged as a pywebview desktop window, built into a single .app / .exe with PyInstaller - no terminal, no setup.

ENGINEERING

The hardest bug: the load cell

1

Problem

The load cell kept dropping to simulated data partway through a session - and once it dropped, every test after it read fake values.

2

Diagnosis

The manual window's live graph and the active test were reading the device at the same time, on different threads. The driver can't be read twice at once, so it corrupted.

3

Fix

Re-architected to a single-owner reader: one background loop owns the device and everything else reads its value. Two simultaneous reads are now impossible, in every window.

TIMELINE

Day by day

Days 1-18 went to clinical data validation (OHSU / HFH cases, the MATLAB pipeline, and site decks). The fixture GUI begins on Day 19.

Day 19 · Jun 10	Annotated the test engine (run_engine) end to end - one code path for simulation and real hardware
Day 20 · Jun 11	Reliable motion + serial - killed the 12 s move lag, stopped false disconnects, fixed the 17 mm reference
Day 21 · Jun 12	Safety UI state - the waiting pill, control locking, safety-stop homes; redid the calibration window
Day 22 · Jun 15	Run management + reporting - made the report numbers match the graphs; the redo-run workflow
Day 23 · Jun 16	Other track - UCI data-parsing script and website planning (no GUI)
Day 24 · Jun 17	Real-hardware behavior - true home each test, Shear wired to the load cell, manual force-feedback
Day 25 · Jun 18	Load-cell reliability - the single-owner reader, plus fatigue and EM safety fixes
Day 26 · Jun 19	Other track - competitor research and a clinical-affairs interview (no GUI)
Day 27 · Jun 22	In progress - snap-to input limits, a formal test plan, and user-story reconciliation

STATUS

Where it stands & what's next

Solid now

- Every test type runs end to end
- Safety behaviors verified in simulation
- Verified on the rig where hardware was available
- Packaged and runnable without a terminal

Next

- Execute the formal test plan (full path coverage)
- Confirm safety-critical cases on the real rig
- Reconcile the user-stories doc to the current GUI

RESOURCES

Documentation & code

User stories

11 features, 100+ stories - every happy and unhappy path, safety handling, and a verification method for each.

[add link]

Test plan

Formal test cases traced to each story, organized by technique and tagged simulation vs real rig.

[add link]

Source code

The full GUI, test engine, and hardware drivers, version-controlled.

github.com/jacqchuang09/test_fixture